

Dokument projektowy

MoodFlow — platforma monitorowania zdrowia psychicznego pracowników

Mateusz Matczuk · Praca inżynierska CDV · 2026

Niniejszy dokument zawiera komplet informacji projektowych dotyczących platformy MoodFlow. Obejmuje: przyjętą metodologię pracy, stack technologiczny rozwiązania, listę wykorzystywanych narzędzi developerskich i kolaboracyjnych, harmonogram pracy w postaci wykresu Gantta z klasyfikacją MoSCoW (wyraźnie wskazującą zakres MVP, funkcje rozszerzające oraz elementy nieobjęte zakresem pracy inżynierskiej), rejestr ryzyk projektowych wraz z planem mitygacji oraz plan monitorowania jakości obejmujący formatkę scenariusza testowego.

Spis treści

1. Metodologia pracy
2. Stack technologiczny rozwiązania
3. Wykorzystywane narzędzia
4. Harmonogram pracy (wykres Gantta z MoSCoW)
5. Ryzyka projektowe i plan mitygacji
6. Plan monitorowania jakości i formatka scenariusza testowego

1. Metodologia pracy

W projekcie MoodFlow zastosowano **metodykę zwinną opartą o hybrydę Scrum-lite oraz Kanban**, dostosowaną do realiów dwuosobowego zespołu studenckiego pracującego nad pracą inżynierską. Wybór metodyki został podyktowany koniecznością łączenia pracy projektowej z innymi obowiązkami akademickimi i zawodowymi, a także potrzebą zachowania elastyczności w reagowaniu na zmiany wymagań formułowanych w trakcie konsultacji z promotorem.

1.1. Iteracje i rytm pracy

Pracę zorganizowano w **dwutygodniowych iteracjach** (sprintach), w ramach których realizowano zamknięty zakres funkcjonalności. Każda iteracja rozpoczyna się od krótkiego planowania (lista zadań na sprint w kolumnie *To do* w tablicy Kanban) i kończy się retrospektywą oraz przeglądem wykonanych zmian. Komunikacja w trakcie sprintu odbywa się asynchronicznie — codzienne aktualizacje statusu w postaci krótkich notatek tekstowych zastępują formalne stand-upy.

1.2. Tablica zadań i przepływ pracy

Zadania są reprezentowane w postaci kart na tablicy Kanban z czterema kolumnami:

- **Backlog** — pomysły i zadania bez przypisanej iteracji;
- **To do** — zadania zaplanowane na bieżący sprint;
- **In progress** — zadania w trakcie realizacji (limit WIP = 2 na osobę);
- **Done** — zadania zakończone, zatwierdzone do scalenia.

1.3. Priorytetyzacja MoSCoW

Zakres funkcjonalności zarządzany jest przy użyciu klasyfikacji **MoSCoW (Must, Should, Could, Won't Have)**. Każda funkcjonalność z PRD otrzymała jeden z czterech priorytetów. Klasyfikacja ta jest źródłem prawdy zarówno dla harmonogramu (kolory na wykresie Gantta w rozdziale 4.), jak i dla decyzji o zakresie MVP. Funkcje oznaczone jako **Won't Have** są jawnie wykluczone z bieżącej iteracji projektu i opisane jako kierunki rozwoju po obronie.

1.4. Definition of Done

Zadanie uznaje się za zakończone, gdy spełnione są łącznie wszystkie poniższe warunki:

- kod został przeglądnęty przez drugiego członka zespołu (Pull Request review);
- linting (ESLint) i typecheck (TypeScript) nie zgłaszają błędów;
- endpointy backendowe pokryte są testem klikalnym lub jednostkowym;
- frontend jest manualnie przetestowany w obu aplikacjach (pracownik + admin);
- zmiana została udokumentowana — w README, w komentarzu PR lub w rozdziale pracy.

1.5. Code review i kontrola wersji

Repozytorium prowadzone jest jako **monorepozytorium Git** z trzema strefami: backend-api, client-frontend, admin-frontend. Każda zmiana wprowadzana jest poprzez gałąź funkcjonalną (*feature branch*) i scalanie do gałęzi głównej odbywa się wyłącznie przez Pull Request. Stosujemy konwencję **Conventional Commits** w języku polskim (np. *fix(client-frontend): wymuś jasne tokeny tekstu na ekranach auth*), co ułatwia zarówno przeglądanie historii, jak i późniejsze opisanie zmian w pracy.

2. Stack technologiczny rozwiązania

Rozwiązanie zostało zaimplementowane jako **trójwarstwowa aplikacja webowa** z osobnymi aplikacjami frontendowymi dla pracownika oraz dla działu HR/administradora i centralnym API obsługującym logikę biznesową. Dobór technologii był uzasadniony przede wszystkim potrzebą bezpieczeństwa danych wrażliwych, czytelnej modularnej architektury oraz możliwością opisania decyzji projektowych w pracy inżynierskiej.

2.1. Warstwa serwerowa (backend-api)

Komponent	Technologia	Uzasadnienie
Framework	NestJS 11	modularna architektura, podział na kontrolery / serwisy / moduły, wbudowane DI
Język	TypeScript 5.9	statyczne typowanie redukuje błędy w warstwie modeli i DTO
ORM	TypeORM 0.3	dojrzała integracja z NestJS, repozytoria + migracje
Baza danych	PostgreSQL 16	relacyjna baza odpowiednia do wielotabelowego modelu RBAC + ankiety + wyniki
Autentykacja	JWT + Passport	tokeny krótkożyciowe + refresh, oddzielenie ról pracownika/HR/ admina
Hashowanie haseł	bcrypt	standard branżowy, odporność na ataki słownikowe
Walidacja DTO	class-validator + class-transformer	deklaratywne reguły walidacji na poziomie kontrolerów
Rate limiting	@nestjs/throttler	ochrona endpointu logowania przed atakami brute-force
Nagłówki HTTP	helmet	podstawowa ochrona przed atakami typu XSS / clickjacking
E-mail	Nodemailer + nestjs-mailer	powiadomienia o nowych testach i statusie zatwierdzenia konta

2.2. Frontend pracownika (client-frontend)

Komponent	Technologia	Uzasadnienie
Framework UI	React 19	komponentowy model UI, ekosystem PWA
Build tool	Vite 8	szybki HMR i krótkie czasy buildu produkcyjnego
Język	TypeScript 5.9	spójność z backendem, typowanie DTO API
Style	Tailwind CSS 3	utility-first, łatwo egzekwowalny design system

State	Redux Toolkit 2	przewidywalny store na sesję i meta-dane użytkownika
Routing	React Router 7	deklaratywne ścieżki, ochrona widoków przez strażników
Animacje	Framer Motion 12	subtelne przejścia, większa płynność UX
Ikony	Lucide React	spójna paleta ikon w stylu line-art
PWA	vite-plugin-pwa	instalowalność, działanie offline po pierwszym wejściu
Powiadomienia	sonner	lekki toast manager

2.3. Frontend administracyjny / HR (admin-frontend)

Komponent	Technologia	Uzasadnienie
Framework UI	Next.js 16 (App Router)	uporządkowana struktura panelu, SSR dla widoków raportowych
Język	TypeScript 5	spójność z backendem i client-frontend
Style	Tailwind CSS 4	ten sam system tokenów co frontend pracownika
Wykresy	Recharts 3	responsywne wykresy linowe i słupkowe do trendów HR
Eksport	jsPDF + html2canvas	eksport raportu HR do PDF po stronie klienta
Ciasteczka	js-cookie	zarządzanie sesją HTTP w panelu
Theming	next-themes	obsługa trybu jasnego/ciemnego
PWA	@ducanh2912/next-pwa	instalowalność panelu HR

2.4. Infrastruktura i wdrożenie

Komponent	Technologia	Uzasadnienie
Konteneryzacja	Docker + Docker Compose	powtarzalne środowisko lokalne i produkcyjne
Środowisko prod	Railway	prosta orkiestracja kontenerów + zarządzana PostgreSQL
System kontroli wersji	Git + GitHub	monorepozytorium, ochrona gałęzi, code review
CI (planowane)	GitHub Actions	lint + typecheck + testy klikalne przy każdym PR

3. Wykorzystywane narzędzia

Zestaw narzędzi został dobrany tak, by zminimalizować tarcia pomiędzy pracą developerską a pracą dokumentacyjną. Większość narzędzi jest darmowa w wariantcie studenckim lub posiada licencje open-source.

3.1. Narzędzia developerskie

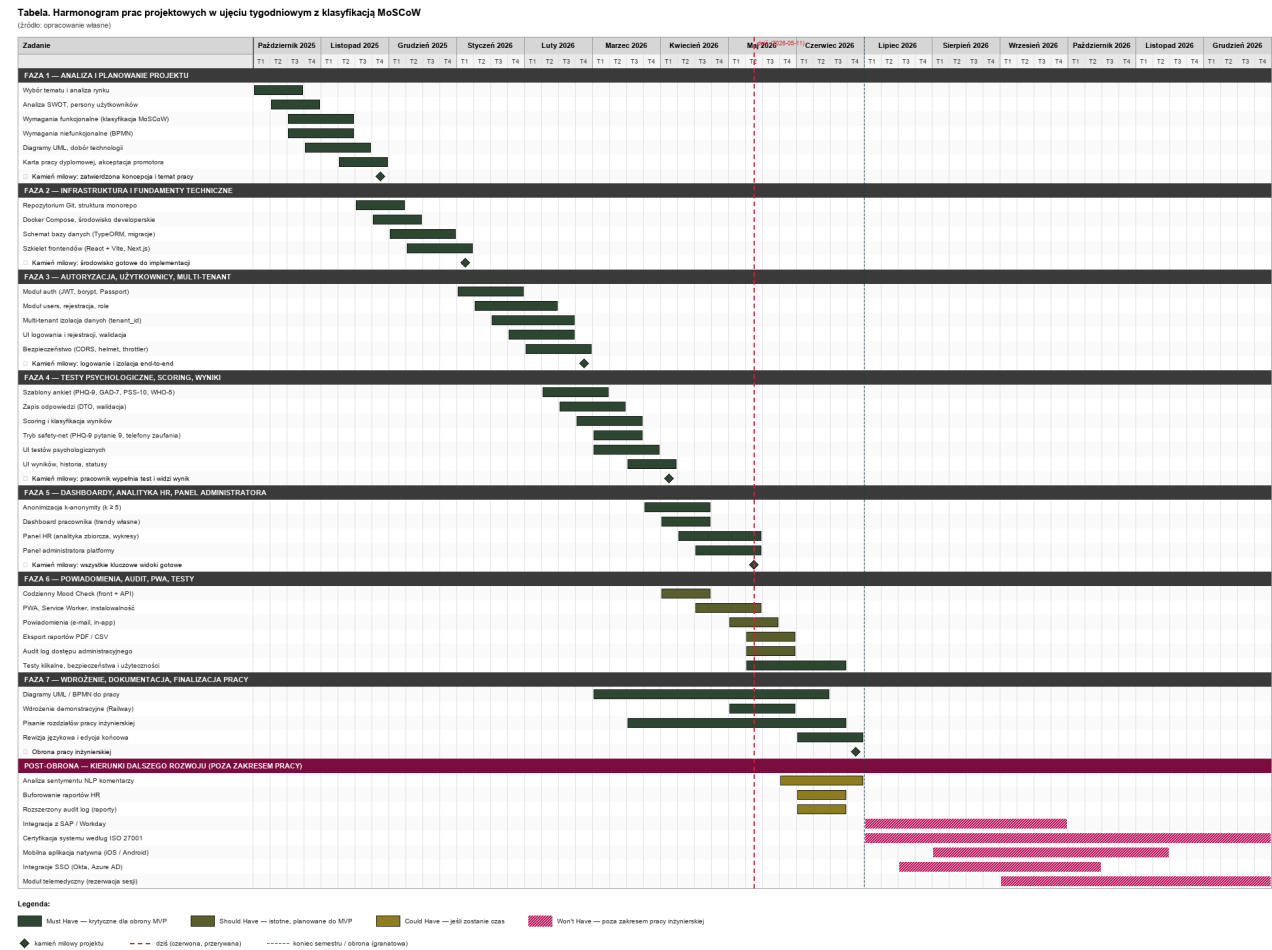
Kategoria	Narzędzie	Zastosowanie
IDE	Visual Studio Code, Cursor	edycja kodu, integracja z AI
Kontener developerski	Docker Desktop	uruchomienie bazy i usług lokalnie
Klient bazy danych	TablePlus / pgAdmin	przeglądanie schematu, ad-hoc query
Klient HTTP	Postman, REST Client (VS Code)	testowanie endpointów API
Linter / formatter	ESLint, Prettier	wymuszanie konwencji kodu
Typecheck	TypeScript Compiler	weryfikacja typów w obu warstwach
Migracje DB	TypeORM CLI	kontrola wersji schematu bazy
Build	Vite, Next.js build, nest build	produkcyjne paczki aplikacji
Testy	Jest (backend), Vitest (front)	testy jednostkowe i integracyjne

3.2. Narzędzia współpracy i dokumentacji

Kategoria	Narzędzie	Zastosowanie
Kontrola wersji	GitHub	PR-y, code review, ochrona gałęzi master
Tablica zadań	GitHub Projects (Kanban)	śledzenie sprintów i statusu zadań
Komunikacja	Discord, Slack	krótkie ustalenia i wymiana plików
Spotkania	Google Meet, MS Teams	konsultacje z promotorem, sync zespołu
Diagramy	draw.io, Mermaid, PlantUML	diagramy ERD, przypadków użycia, sekwencji
Design UI	Figma, Pencil	makiety i prototypy, biblioteka komponentów
Pisanie pracy	Microsoft Word, python-docx	rozdziały pracy, generowanie tabel i listingów
Notatki / wiedza	Notion, Obsidian	notatki badawcze, decyzje architektoniczne
Wsparcie AI	Claude Code, ChatGPT	review kodu, generowanie fragmentów dokumentacji
Zarządzanie hasłami	1Password	bezpieczne przechowywanie sekretów rozwojowych

4. Harmonogram pracy (wykres Gantta z MoSCoW)

Poniżej przedstawiono harmonogram pracy w postaci wykresu Gantta z naniesioną klasyfikacją **MoSCoW**. Każde zadanie ma kolor odpowiadający przypisanemu priorytetowi: **zielony** — Must Have, **oliwkowy** — Should Have, **musztardowy** — Could Have, **karminowy z ukośnym wzorem** — Won't Have. Czerwona, przerywana linia pionowa oznacza datę dzisiejszą (2026-05-11). Kropkowana, granatowa linia oznacza umowny koniec semestru (czerwiec 2026) — okres obrony pracy inżynierskiej.



Rys. 1. Harmonogram pracy MoodFlow z klasyfikacją MoSCoW.

4.1. Funkcjonalności Won't Have — uzasadnienie wykluczenia

Cztery elementy z dolnej części wykresu (kolor karminowy z ukośnym wzorem) zostały świadomie wykluczone z zakresu pracy inżynierskiej. Ich realizacja wykraczałaby poza ramy czasowe semestru oraz poza zakres uzasadniony charakterem pracy akademickiej. Funkcje te zostały opisane jako rozszerzenia w sekcji „Kierunki dalszego rozwoju” pracy inżynierskiej:

- Integracja z systemami HR (SAP, Workday)** — wymaga zakupu licencji i dostępu do środowisk produkcyjnych klientów; brak wartości naukowej.
- Certyfikacja systemu według ISO 27001** — proces wielomiesięczny, wymaga audytu zewnętrznego; zakres pracy ogranicza się do zgodności z dobrymi praktykami.
- Mobilna aplikacja natywna (iOS/Android)** — PWA zapewnia większość korzyści mobilnych; aplikacje natywne wymagałyby drugiej linii kodu.
- Integracje SSO (Okta, Azure AD)** — wymaga dostępu do tenantów korporacyjnych; w MVP wystarczy autentykacja własna.

- **Moduł telemedyczny (rezerwacja sesji terapeutycznych)** — wymaga partnerstw z platformami telemedycznymi i regulacji prawnych specyficznych dla medycyny.

5. Ryzyka projektowe i plan mitygacji

Rejestr ryzyk powstał w fazie analizy projektu i jest okresowo aktualizowany. Każde ryzyko zostało ocenione w skali trzystopniowej (Niskie / Średnie / Wysokie) w wymiarach prawdopodobieństwa wystąpienia oraz wpływu na powodzenie projektu. Dla każdego ryzyka określono działania mitygujące oraz osobę odpowiedzialną.

#	Ryzyko	P	W	Mitygacja
R1	Brak czasu na implementację wszystkich modułów MVP	W	W	Twarda priorytetyzacja MoSCoW; skupienie na Must Have. Funkcje Could Have realizowane wyłącznie po zamknięciu MVP.
R2	Wyciek danych wrażliwych dotyczących zdrowia psychicznego	N	Kr	Multi-tenant izolacja w warstwie ORM (tenant_id), audyt logów dostępu, k-anonimity, minimalizacja danych.
R3	Identyfikacja pojedynczego pracownika w raporcie HR	Ś	W	K-anonimity ($k \geq 5$) wymuszone po stronie API; agregacja, brak ujawniania komentarzy w postaci wrażliwej.
R4	Niewystarczająca wydajność panelu HR przy dużej liczbie wyników	Ś	Ś	Indeksy w bazie, paginacja, lazy-loading komponentów wykresów, buforowanie agregatów (Could Have).
R5	Niezgodność z RODO w warstwie procesowej (klauzule, retencja)	Ś	W	Konsultacja z opiekunem ds. prywatności w organizacji pilotażowej; checklisty zgodne z UODO.
R6	Sygnał ryzyka samobójczego pominięty przez system	N	Kr	Zawsze włączony tryb safety-net dla PHQ-9 pytanie 9 > 0; dedykowany komunikat + telefony zaufania.

R7	Awaria środowiska produkcyjnego (Railway)	Ś	Ś	Konfiguracja zdublowana w docker-compose, prowadzenie kopii zapasowych bazy, dokumentacja kroków odtworzenia.
R8	Konflikt wersji zależności (NestJS 11, React 19, Next 16)	N	Ś	Zamrożone wersje w package-lock.json, sprawdzanie kompatybilności przy każdej aktualizacji.
R9	Utrata danych w trakcie developmentu	Ś	Ś	Migracje TypeORM przed każdą zmianą schematu, regularne commity, wolumeny Docker na dysku.
R10	Niedostępność członka zespołu (choroba, sesja egzaminacyjna)	Ś	W	Praca w parze, dokumentacja decyzji w README, każdy moduł posiada drugiego „opiekuna”.
R11	Zmiana wymagań ze strony promotora w trakcie semestru	Ś	Ś	Cotygodniowe konsultacje, dokumentacja decyzji w pliku decisions.md, elastyczna architektura modularna.
R12	Brak walidowanych skal psychologicznych w domenie publicznej	N	W	Dobór skal z domeny publicznej (PHQ-9, GAD-7, PSS-10, WHO-5), źródła zacytowane w pracy.
R13	Słaba użyteczność interfejsu pracownika (porzucanie ankiet)	Ś	Ś	Testy klikalne z 8–10 osobami, kwestionariusz SUS, iteracja UI przed wdrożeniem.
R14	Niespójność designu między aplikacjami pracownika i HR	Ś	N	Wspólny token system Tailwind, biblioteka komponentów wzorowana na Figma, audyt UI raz na sprint.

R15	Brak akceptacji konta firmy z powodu niejasnego procesu	N	Ś	Komunikaty statusu w panelu, e-mail powiadamiający o akceptacji, instrukcja w README.
-----	--	---	---	--

Legenda: *P* = prawdopodobieństwo wystąpienia (*N* – niskie, *Ś* – średnie, *W* – wysokie). *W* = wpływ na powodzenie projektu (*N* – niski, *Ś* – średni, *W* – wysoki, *Kr* – krytyczny).

6. Plan monitorowania jakości i formatka scenariusza testowego

6.1. Cele monitorowania jakości

Celem planu jakości w projekcie MoodFlow jest zapewnienie, że dostarczane funkcjonalności są: poprawne (zgodne ze specyfikacją), bezpieczne (chronią dane wrażliwe), użyteczne (umożliwiają pracownikowi szybkie wypełnienie ankiety) i wydajne (czas odpowiedzi API w rozsądnym przedziale). Monitorowanie jakości prowadzone jest w sposób ciągły — równoległe z developmentem — a nie jako wydzielony etap końcowy.

6.2. Typy testów

- **Testy jednostkowe (unit)** — Pokrywają logikę domenową w izolacji od I/O. Realizowane głównie dla scoringu testów psychologicznych oraz funkcji anonimizacji. Framework: Jest (backend) i Vitest (front).
- **Testy integracyjne** — Pokrywają warstwę kontrolerów + serwisów + ORM. Uruchamiane na lokalnej instancji PostgreSQL w kontenerze. Sprawdzają poprawność scenariuszy multi-tenant, autoryzacji i walidacji DTO.
- **Testy klikalne (E2E manualne)** — Pełne ścieżki użytkownika przechodzone ręcznie według scenariuszy opisanych w dokumencie 5.1 (Testy klikalne). Każdy scenariusz zawiera prerequisites, kroki, oczekiwany wynik i kryterium akceptacji.
- **Testy bezpieczeństwa** — Próby obejścia autoryzacji (token innej roli, manipulacja tenant_id), próby SQL-injection (zachowanie po stronie ORM), test rate-limitingu na endpoint logowania.
- **Testy wydajnościowe** — Pomiar czasu odpowiedzi krytycznych endpointów (logowanie, zapis odpowiedzi, agregaty HR) narzędziem ApacheBench oraz audyt Lighthouse dla widoków frontowych.
- **Testy użyteczności** — Sesje z 8–10 użytkownikami zewnętrznymi (znajomi, rodzina), obserwacja porzuceń, kwestionariusz SUS po sesji. Wyniki zostaną opisane w rozdziale „Testy i wyniki”.

6.3. Środowiska testowe

Środowisko	Cel	Baza danych	Adres / dostęp
Lokalne (dev)	codzienna praca developerska	PostgreSQL w Dockerze (port 5432)	localhost:3000 / :3001 / :4000
Stage (Railway)	demo dla promotora i obrona	PostgreSQL managed (Railway)	domena Railway HTTPS
Testy klikalne	scenariusze E2E manualne	kopia stage z zsynchronizowanymi seedami	domena Railway / dev

6.4. Kryteria akceptacji i metryki jakości

Metryka	Cel	Sposób pomiaru
Czas odpowiedzi API (P50)	≤ 300 ms dla zapytań standardowych	Logi NestJS + ApacheBench

Czas odpowiedzi API (P95)	≤ 1 s	ApacheBench (1000 req)
Lighthouse Performance (front)	≥ 90 (mobile)	Lighthouse w Chrome DevTools
Lighthouse Accessibility	≥ 95	Lighthouse w Chrome DevTools
Pokrycie scoringu testami	≥ 80%	Jest --coverage
Liczba błędów 5xx w trakcie testów klikalnych	0	Logi backendu + scenariusze E2E
Skuteczność blokady multi-tenant	100%	Scenariusz bezpieczeństwa #SEC-01
SUS — System Usability Scale	≥ 70 (akceptowalne)	Kwestionariusz po sesji UX

6.5. Formatka scenariusza testowego

Każdy scenariusz testu klikalnego dokumentowany jest według poniższej formatki. Formatka ta jest spójna ze strukturą rozdziału 5.1. pracy inżynierskiej i pozwala jednoznacznie odtworzyć test, zweryfikować rezultat oraz przypisać go do wymagania funkcjonalnego.

Pole	Opis
ID scenariusza	Unikalny identyfikator (np. TC-AUTH-001, TC-HR-014)
Tytuł	Krótki opis testowanej ścieżki użytkownika
Powiązane wymaganie	Numer wymagania z rozdziału 2.4 (np. WF-12)
Priorytet MoSCoW	Must / Should / Could / Won't
Aktor	Pracownik / HR / Administrator firmy / Administrator platformy
Środowisko	dev / stage / lokalne
Warunki wstępne	Stan systemu przed testem (np. konto zatwierdzone, kod firmy znany)
Dane testowe	Wartości używane podczas testu (loginy, hasła, kod firmy, odpowiedzi)
Kroki	Numerowana lista czynności wykonywanych przez testera
Oczekiwany wynik	Co system powinien wyświetlić / zapisać / wywołać
Kryterium akceptacji	Warunek konieczny do uznania scenariusza za zaliczony
Wynik faktyczny	Co zaobserwowano podczas wykonania testu
Status	PASS / FAIL / BLOCKED / SKIPPED
Uwagi / błędy	Numer zgłoszenia, screen, dodatkowe obserwacje
Data wykonania	RRRR-MM-DD
Wykonujący	Imię i nazwisko testera

6.6. Przykład wypełnionej formatki

Poniżej przykład wypełnionej formatki dla scenariusza krytycznego z punktu widzenia bezpieczeństwa: weryfikacja, że konto HR firmy A nie ma dostępu do wyników firmy B.

Pole	Wartość
ID scenariusza	TC-SEC-003
Tytuł	Brak dostępu HR firmy A do wyników firmy B (multi-tenant)
Powiązane wymaganie	WN-SEC-04 (izolacja danych pomiędzy tenantami)
Priorytet MoSCoW	Must Have
Aktor	HR firmy A (token JWT z tenant_id = A)
Środowisko	stage (Railway)
Warunki wstępne	W systemie istnieją dwie firmy A i B z różnymi tenant_id. Każda posiada minimum 5 zatwierdzonych pracowników z wynikami PHQ-9.
Dane testowe	Konto HR firmy A: hr@firmaA.test / hasło testowe; URL endpointu raportu firmy B: /api/v1/analytics/wellbeing-index?tenantId=B
Kroki	1. Zaloguj się jako HR firmy A. 2. Skopiuj token JWT z DevTools. 3. Wywołaj endpoint Postmanem z parametrem tenantId=B (firma obca). 4. Zaobserwuj odpowiedź API.
Oczekiwany wynik	API zwraca odpowiedź 403 Forbidden z komunikatem „Insufficient permissions” oraz nie zwraca jakichkolwiek danych firmy B w treści odpowiedzi.
Kryterium akceptacji	HTTP 403 + pusty body danych firmy B + wpis w audit_logs z akcją „forbidden_cross_tenant_read”.
Wynik faktyczny	API zwróciło 403 Forbidden, body nie zawiera danych firmy B, w audit_logs pojawił się wpis o próbie dostępu.
Status	PASS
Uwagi / błędy	—
Data wykonania	2026-05-09
Wykonujący	Mateusz Matczuk

6.7. Sposób raportowania wyników

Wyniki wszystkich scenariuszy zbierane są w jednym arkuszu zbiorczym, zawierającym kolumny: ID scenariusza, status (PASS/FAIL/BLOCKED/SKIPPED), datę wykonania, wykonującego oraz odniesienie do ewentualnego zgłoszenia. Po każdej iteracji sprintu generowany jest raport pokrycia testowego, który stanowi podstawę decyzji o gotowości iteracji do scalenia do gałęzi głównej i, w konsekwencji, do wdrożenia demonstracyjnego.

Scenariusze, które uzyskały status **FAIL**, są opisywane jako zgłoszenia w GitHub Issues z etykietą *type:bug* i priorytetem zgodnym z klasyfikacją MoSCoW funkcji, której dotyczą. Naprawa zgłoszenia skutkuje ponownym wykonaniem scenariusza i aktualizacją statusu w arkuszu zbiorczym.